

现在需要你帮我调研一下 MOPD 类似的工作，以及我如果想做这方面的创新，我可以从哪些方面入手？用来发文章哈！

Q1

先给结论：MOPD 值得做，但赛道已经从“提出多教师蒸馏”快速进入“诊断 MOPD 为什么失效、怎样稳定融合”的阶段。现在再做简单的多教师加权、按领域采样或 Top-K 蒸馏，创新性已经不够。最值得押注的是：

- 1. 摆脱 oracle domain label 的动态教师路由；
- 2. 从 token 数量平衡升级到“行为影响力”平衡；
- 3. 解决异源、跨模型家族教师的稳定融合；
- 4. 用 Pareto/约束优化保证每项能力不退化。

以下按 Multi-Teacher On-Policy Distillation 理解。另有一篇同名的 Multi-Rollout OPD，后面会单独说明。

1. MOPD 到底解决什么问题

MOPD 的基本流程是：

通用 SFT → 各领域独立 RL 得到 Math/Code/IF/SWE 教师 → 学生自己生成轨迹 → 按领域路由到对应教师 → 用 token-level reverse KL 蒸馏。

可写成：

$$\mathcal{L}_{\text{MOPD}} = \sum_d w_d \mathbb{E}_{x \sim D_d, y \sim \pi_\theta} \left[\frac{1}{|y|} \sum_t D_{\text{KL}} \left(\pi_\theta(\cdot | x, y_{<t}) \parallel \pi_{T_d}(\cdot | x, y_{<t}) \right) \right].$$

核心优势是学生在自己的状态分布上接受教师纠错，避免普通 teacher-SFT 的 exposure bias。

你现有目录主要覆盖单教师 OPD 主线，例如 MiniLLM 2306.08543_MiniLLM.pdf、Veto 2601.07155_Veto.pdf、Entropy-Aware OPD 2603.07079_Entropy-Aware_OPD.pdf、Demystifying OPD 2604.08527v1_DemystifyingOPD.pdf 和 TIP 2604.14084_TIP.pdf。但 2026 年 6—8 月的几篇 MOPD 直接竞品还不在于目录中，建议优先补齐。

2. 相关工作谱系

主线	代表工作	与 MOPD 的关系
单教师 OPD 奠基	GKD 、 MiniLLM	建立“student samples, teacher scores”的基本范式
工业级多领域蒸馏	MiMo-V2-Flash 、 Nemotron-Cascade 2	实际使用多个领域教师恢复和整合能力
正式 MOPD 方法	MOPD	系统比较 Mix-RL、Cascade RL、Off-policy FT、Param-Merge；在 Qwen3-30B-A3B 上达到 0.937 normalized score
通用能力恢复	CaMOPD	研究领域能力与通用能力梯度相互抵消，使用交替训练和 gap-based selection

主线	代表工作	与 MOPD 的关系
多平台/Agent 应用	UI-MOPD	将桌面和移动 GUI 教师整合进统一 Agent
长视野 Agent OPD	TCOD 、 SOD	分别处理跨轮误差累积和 step-level 教师信号失真
Top-K 行为漂移	When Top-K Misses the Decision	即使 Top-32 覆盖 99.99% 概率质量，也可能遗漏 <code><tool_call></code> 这种决定行为模式的关键 token
泛化和跨域干扰	Every Coin Has Two Sides	发现教师影响不能被 prompt routing 完全限制，多教师混合会产生 mixture-dependent seesaw
最新开放诊断	Open-MOPD	发现朴素 MOPD 只恢复 35.6% 能力空间；通过 token-share balancing、gap allocation、reward refresh 提升到 83.4%
同名但不同问题	Multi-Rollout OPD	利用同一问题的成功/失败 peer rollouts 改善单教师信号，不是多领域教师融合

Open-MOPD 是现在必须对标的工作。它已经证明：

- IF 虽占约 20% prompts，却可能只占约 1% gradient tokens；
- Math/Code 平均响应约 10,500 tokens，IF 只有约 409 tokens；
- 单纯 prompt 均匀采样并不等于优化预算均匀；
- 在其 oracle routing 设定下，主要问题不是梯度冲突，而是 token 预算、动态收敛速度和 stale reward。[论文与开放配方](#)

3. 最值得做的创新方向

A. 动态教师仲裁：我最推荐

现有 MOPD 几乎都假设“每个 prompt 的领域标签已知，而且只对应一个正确教师”。真实任务经常是 Math+Code、IF+Tool、Search+Reasoning 的组合任务。

可以做：

Beyond Oracle Routing: Utility-Aware Multi-Teacher On-Policy Distillation

核心创新：

- 不再按固定 domain label 路由；
- 同时考虑教师能力和教师—学生分布兼容性；
- 学习 prompt-level 或 step-level sparse router；
- 当教师不可靠时允许 abstain，退回 verifier RL 或学生自身策略；
- 对组合任务选择 Top-2 教师并做动态权重融合。

一个合理的教师效用定义是：

$$u_d(x) = R_d(\hat{y}_d) - \lambda D_{\text{KL}}(\pi_S \parallel \pi_{T_d}),$$

即教师在当前样本上既要“做得好”，又不能离学生分布远得无法学习。

文章故事会很完整：

1. 证明 oracle routing 是隐藏假设；
2. 证明在 noisy label 和 composite prompt 下 Open-MOPD 仍会失败；
3. 提出 competence-compatibility router；
4. 在等教师推理预算下超过 hard routing、uniform mixture 和 confidence routing。

创新性高，但实现难度也较高。

B. 行为影响力平衡，而非 token 数量平衡

Open-MOPD 平衡的是 token share，但不同 token 对最终行为的影响可能相差几个数量级。例如 `<tool_call>`、EOS、代码围栏、答案格式和 action type token 都可能控制整段生成模式。

可提出：

Leverage-MOPD: Balancing Behavioral Influence in Multi-Teacher Distillation

核心假设：

$\text{prompt share} \neq \text{token share} \neq \text{behavioral influence share}$.

方法上可以：

- 自动检测 behavior-switch positions；
- 根据局部干预后最终行为概率变化估计 token leverage；
- 对高影响且教师冲突的 token 做 support union、软裁剪或约束更新；
- 对普通推理 token 使用 Open-MOPD 的 token-share balancing。

这条线能自然连接 Open-MOPD 和 Top-K tool-call drift，适合写成“机制发现 + 方法修复”型文章。

C. 异源/跨家族教师的稳定 MOPD

原始 MOPD 明确发现：换成更强但异源的教师后，初始 KL 约增大 5 倍，训练可能坍缩甚至发散。[原始消融](#)

可以研究：

- 学生—教师之间的自适应 bridge distribution；
- 根据 support overlap 动态增大教师权重；
- off-policy cold start $\rightarrow$ bridge OPD $\rightarrow$ full MOPD 的课程；
- 多个不同 tokenizer、不同推理风格教师共同蒸馏。

不过跨 tokenizer 本身已经很拥挤，包括 [SimCT](#) 和 [Byte-Prefix Marginalization](#)。因此论文必须强调“多个异源教师如何稳定仲裁”，不能只做 token 对齐。

D. Pareto-safe MOPD：相对稳妥、最好落地

目前方法通常优化平均 normalized score，但平均分提高仍可能掩盖某个领域退化。

可以把问题写成多目标约束优化：

$$\max_{\theta} \min_d \Delta R_d(\theta), \quad \text{s.t.} \quad R_d(\theta) \geq R_d(\theta_0) - \epsilon_d.$$

方法上使用：

- 每领域 verifier reward；
- OPD 作为 dense guidance；
- primal-dual 动态领域权重；
- 对退化领域自动提高 Lagrange multiplier；
- 同时优化 average recovery 和 worst-domain recovery。

它与 Open-MOPD 的区别是：Open-MOPD 根据 teacher-student reward magnitude 分配预算；你根据真实任务能力和非退化约束分配预算。这个方向计算可控，理论也容易写。

E. 低成本/主动式 MOPD

MOPD 需要每条学生轨迹经过教师 prefill，教师数量增加后成本很高。可以研究：

- 只在高熵、高分歧或 behavior-critical token 查询教师；
- 学习 teacher-query policy；
- 缓存教师 logits，并检测何时失效；
- 在固定 teacher FLOPs 下最大化 capability recovery。

但 TIP 已经证明保留约 50% 高熵 token 可以接近全 token 训练，所以必须做成“多教师查询与路由联合优化”，单纯 token 筛选不够新。

4. 我建议的具体论文方案

首选题目：

Beyond Oracle Routing: Competence-Compatibility Arbitration for Multi-Teacher On-Policy Distillation

实验设计：

- 基础配方：直接复用 Open-MOPD 的 SmoLLM3-3B、Math/Code/IF 教师和数据；
- 三种设置：
 1. 单领域、正确标签；
 2. 10%/20%/30% noisy routing；
 3. Math+IF、Code+IF、Tool+IF 组合任务；
- Baselines：
 - Mix-RL；
 - Naive MOPD；
 - 原始 MOPD；
 - Open-MOPD；

- uniform teacher mixture;
 - confidence routing;
 - CaMOPD;
 - oracle per-example teacher;
- 主要指标:
 - average normalized recovery;
 - worst-domain recovery;
 - domain score 方差/极差;
 - composite-task success;
 - OOD 能力保持;
 - teacher forward 次数和总 FLOPs;
 - routing accuracy 只能作为辅助指标, 不能代替最终任务表现;
- 必做消融:
 - competence only vs. compatibility only vs. 二者结合;
 - hard Top-1 vs. soft Top-2;
 - prompt-level vs. step-level routing;
 - same-origin vs. cross-origin;
 - 有无 Open-MOPD 三个 balancing 组件;
 - 等 teacher-compute 比较。

如果算力不足, 不建议从头训练三个 RL 教师。可以直接用 Open-MOPD 发布的 Stage-1/Stage-2 checkpoints, 把论文聚焦在 Stage-3 integration, 这样最省算力, 也最容易与最新结果公平比较。Open-MOPD 完整复现实验使用的是 8xA100-80GB, 但只做集成阶段会轻很多。

5. 现在尽量不要做的题目

- 只做不同领域采样比例: Open-MOPD 已基本覆盖。
- 只做梯度冲突消除: Open-MOPD 在 oracle routing 下发现它并非主要瓶颈, CaMOPD 也已做相关分析。
- 只把 Top-K 从 16 改成 32/64: 原始 MOPD、TIP 和 tool-call drift 已覆盖。
- 只把 MOPD 应用到一个新领域: 除非有新数据集和新 failure mode, 否则容易被评价为工程迁移。
- 只加入 entropy weighting、correct/incorrect routing 或普通 RL reward: Veto、Entropy-Aware OPD、SCOPE、SRPO、HDPO 等单教师工作已经非常拥挤。
- 只做 cross-tokenizer 对齐: 2026 年已经出现多篇直接工作。

综合新颖度、可实现性和文章完整度, 我的排序是:

1. 动态教师仲裁与组合任务;
2. 行为影响力平衡;
3. Pareto-safe、verifier-aware MOPD;
4. 异源多教师稳定融合;
5. 低成本主动查询 MOPD。

Q2:

文档中的内容是你某次回答的结果。我觉得你提出的《最推荐的方向：Externality-Aware MOPD》idea还是可以做的，但是我不知道具体的可行性？？？

方向 核心做法可行性 与已有工作的重合风险

Transfer-aware dynamic sampling 学习 $D \times D$ 能力影响矩阵，动态分配 rollout 高 低—中

Fragile-capability replay 每次更新后，自动回放预计受损最大的其他领域样本 很高 中

Progress-aware sampling 按“单位 token 实际进步+遗忘速度”采样，而非 raw gap 高 中—高

这几个方向你都觉得可行性很高，但是我完全不知道怎么做？所以你能不能再详细的解释一下呢？同时，我看你列出的可参考的论文里，也有类似的任务使用了类似的方法，所以你还是得具体得详细解释一下。我才知道是否真的可行！

可以做，而且真正可行的版本并没有听起来那么复杂。最重要的一点是：

“学习 $D \times D$ 影响矩阵”不再是训练一个神经网络，而是在线维护 D^2 个统计量。
如果只有 Math、Code、Instruction Following 三个领域，那么矩阵里只有 9 个数。

我建议把三个想法组织成一篇论文，而不是三篇：

- 主方法：Transfer-aware dynamic rollout allocation
- 保护机制：Fragile-capability replay
- 辅助信号/消融：Progress-aware sampling

暂定题目可以是：

Externality-Aware MOPD: Cross-Domain-Aware Rollout Allocation for Multi-Teacher On-Policy Distillation

一、先用一个例子解释“externality”到底是什么

假设学生模型同时向三个老师蒸馏：

- 数学老师
- 代码老师
- 指令遵循老师

传统 gap-aware sampling 看到：

领域	Teacher-student gap
Math	0.70
Code	0.30
IF	0.20

它很自然会认为：

Math gap 最大，下一批多生成 Math rollout。

但问题是：用 Math 更新学生之后，可能发生：

- Math 提升很多;
- Code 稍微提升;
- Instruction Following 明显下降。

我们将一次 Math 更新对所有能力的影响记为：

训练领域 ↓ / 被影响领域 →	Math	Code	IF
Math	+0.12	+0.03	-0.08
Code	+0.02	+0.09	-0.02
IF	-0.01	+0.01	+0.10

这里：

- +0.12: 训练 Math 能让 Math probe loss 下降;
- -0.08: 训练 Math 会让 IF probe loss 上升;
- 每个数都可以按每 1000 个 rollout token 归一化。

这就是 externality: 训练一个领域，不只影响这个领域自己，也会影响其他领域。

如果 IF 当前已经在快速遗忘，我们把 IF 的重要性设成 2，其他能力设成 1，那么三类数据的综合价值为：

$$U_{\text{Math}} = 0.12 + 0.03 + 2 \times (-0.08) = -0.01 \quad [U_{\text{Code}} = 0.02 + 0.09 + 2 \times (-0.02) = 0.07] \quad [U_{\text{IF}} = -0.01 + 0.01 + 2 \times 0.10 = 0.20]$$

此时虽然 Math gap 最大，系统也不会立刻把大部分算力给 Math，因为它对 IF 的伤害太大。

它可能采用如下 rollout 比例：

$$p_{\text{Math}} = 0.4, \quad p_{\text{Code}} = 0.2, \quad p_{\text{IF}} = 0.4$$

这组分配对 IF 的预计影响是：

$$-0.08 \times 0.4 - 0.02 \times 0.2 + 0.10 \times 0.4 = +0.004$$

也就是说：一边学 Math，一边用 IF rollout 抵消其副作用。

这就是最核心的新想法：

不再只问“哪个领域最需要学习”，而是问“训练哪个领域，在当前模型状态下，对所有领域的净收益最大”。

二、Transfer-aware dynamic sampling 具体怎么做？

2.1 先定义影响矩阵

假设所有目标都写成需要最小化的 loss。

训练领域 i 的 MOPD 梯度：

$$g_i = \nabla_{\theta} L_i^{\text{MOPD}}(\theta)$$

领域 j 的 probe loss 梯度:

$$h_j = \nabla_{\theta} L_j^{\text{probe}}(\theta)$$

如果用领域 i 做一步更新:

$$\theta' = \theta - \eta g_i$$

那么领域 j 的 loss 变化近似为:

$$L_j(\theta') - L_j(\theta) \approx -\eta h_j^{\top} g_i$$

因此定义:

$$A_{ij} = \eta h_j^{\top} g_i$$

解释如下:

- $A_{ij} > 0$: 训练 i 预计会改善 j ;
- $A_{ij} < 0$: 训练 i 预计会损害 j ;
- A_{ii} : 领域自身的学习收益;
- $A_{ij}, i \neq j$: 跨领域迁移或干扰。

特别注意: 这里不一定要用 cosine。最初可以同时记录:

$$\cos(g_i, h_j)$$

和

$$h_j^{\top} g_i$$

cosine 只表示方向是否一致, 但丢掉了更新幅度; 真正做资源分配时, 经过归一化的 dot product 通常更有意义。

2.2 Probe bank 是什么?

每个领域准备一个很小的 held-out prompt 集合:

$$V_{\text{math}}, V_{\text{code}}, V_{\text{IF}}$$

例如每个领域 64 个 prompt。

这些 prompt:

- 不参与正式训练;
- 不直接使用测试集;
- 每隔一段时间, 用当前 student 重新生成 rollout;
- 使用该领域对应的 teacher 计算 MOPD surrogate loss;
- 最终 benchmark accuracy 只用来验证, 不用来求梯度。

为什么要重新生成 rollout?

因为 MOPD 是 on-policy distillation。学生策略在变化, 如果一直使用很早以前的 completion, 估计就会越来越 stale。

顺便澄清一个容易混淆的地方:

MOPD 中通常是 student 产生 rollout, teacher 为 student trajectory 提供 token-level 信号。你要修改的是 rollout 前的 prompt/domain sampler, 不是让 teacher 替 student rollout。

2.3 最低成本的实现方法

不要逐样本计算完整梯度。先做领域级估计。

假设每隔 $H = 100$ 个训练 step 更新一次矩阵:

1. 从 Math、Code、IF 各取一个小 calibration batch。
2. 分别计算三个训练梯度 $g_{\text{math}}, g_{\text{code}}, g_{\text{IF}}$ 。
3. 在三个 probe bank 上计算 $h_{\text{math}}, h_{\text{code}}, h_{\text{IF}}$ 。
4. 计算 9 个 gradient inner product。
5. 用 EMA 平滑矩阵。

$$A_t = \rho A_{t-H} + (1 - \rho) \hat{A}_t$$

例如 $\rho = 0.9$ 。

伪代码大致是:

```
if step % control_interval == 0:
    refresh_probe_rollouts(student)

    train_grads = {}
    probe_grads = {}

    for domain in domains:
        train_grads[domain] = grad(
            mopd_loss(calibration_batch[domain]),
            selected_parameters
        )

    for domain in domains:
        probe_grads[domain] = grad(
            probe_loss(probe_bank[domain]),
            selected_parameters
        )

    for source in domains:
        for target in domains:
            predicted_gain = lr * dot(
                train_grads[source],
                probe_grads[target]
            )
            influence[source, target] = ema(
                influence[source, target],
                predicted_gain
            )
```

梯度到底在哪些参数上计算？

这是可行性最关键的地方。

不推荐一开始计算全模型逐样本梯度。可以按以下顺序实验：

1. LoRA 参数；
2. 最后一个 Transformer block；
3. 最后两层加 LM head；
4. 分层 gradient sketch；
5. 最后才考虑全参数。

如果只在 LoRA 参数或最后一层上计算，3 个领域意味着每个控制周期大约多做 6 次小规模 backward，而不是给每个候选 prompt 单独 backward。

所以这个方法是：

- 领域级：可行；
- 小批次级：基本可行；
- 逐样本全参数级：成本过高，不推荐。

2.4 如何把矩阵变成 rollout 比例？

最简单的方法是计算每个训练领域的综合收益：

$$U_i = \frac{\sum_j w_j A_{ij}}{C_i}$$

其中：

- A_{ij} ：训练领域 i 对能力 j 的预测影响；
- w_j ：当前能力 j 的重要性；
- C_i ：领域 i 的 rollout 成本，例如平均 token 数。

权重可以设置为：

$$w_j = 1 + \lambda G_j + \mu F_j$$

其中：

- G_j ：当前 teacher-student gap；
- F_j ：当前遗忘速度。

然后得到采样比例：

$$p_i = (1 - \epsilon) \frac{\exp(U_i/\tau)}{\sum_k \exp(U_k/\tau)} + \frac{\epsilon}{D}$$

ϵ 是 exploration floor，防止某个领域永远不再被采样。

正式版本还可以加入 no-regression constraint：

$$\sum_i p_i A_{ij} \geq -\delta_j, \quad \forall j$$

意思是：

下一轮 rollout 分配不能让任何领域的预计退化超过阈值。

因为只有三个领域，这其实只是一个很小的线性规划问题，不需要训练强化学习 scheduler。

2.5 应该修改 MOPD 代码的什么位置？

最小改动是替换 prompt sampler：

```
# 原来
domain = random.choice(domains)

# 修改后
domain = categorical_sample(current_rollout_ratio)

prompt = sample_prompt(domain)
response = student.generate(prompt)
teacher_signal = teachers[domain].score(prompt, response)

loss = open_mopd_loss(response, teacher_signal)
optimizer.step()
```

也就是说：

- Student rollout 逻辑不变；
- Teacher routing 不变；
- Open-MOPD loss 不变；
- reward refresh 不变；
- 只修改“下一批从哪个领域抽 prompt”。

因此它与 Open-MOPD 是互补关系。Open-MOPD 主要解决 token-share、gap-driven budget 和 reward staleness；你的方法在更外层决定下一窗口应生成哪些领域的 rollout。 [Open-MOPD](#)

三、Fragile-capability replay 具体怎么做？

这个方向不是“把所有旧数据再训练一次”，而是：

当前准备训练 Math 时，先预测哪些 Code/IF prompt 会被这次更新伤害，然后只回放这些最脆弱的 prompt。

3.1 最可行的实现：virtual update

假设当前 batch 来自 Math。

第一步，计算 Math 梯度：

$$g_{\text{math}} = \nabla_{\theta} L_{\text{math}}$$

第二步，构造一个不真正写回模型的虚拟参数：

$$\theta_{\text{virtual}} = \theta - \eta g_{\text{math}}$$

第三步，对其他领域的 probe 样本 z 计算：

$$d_z = L_z(\theta_{\text{virtual}}) - L_z(\theta)$$

其中：

- $d_z > 0$: 该样本预计受损；
- $d_z < 0$: 该样本预计会受益。

选出最大的 Top-K:

$$\mathcal{R} = \text{TopK}_z(d_z)$$

然后把这些 prompt 加入当前或下一次 MOPD batch。

伪代码：

```
source_loss = mopd_loss(math_batch)
source_grad = grad(source_loss)

virtual_student = make_virtual_update(
    student,
    source_grad,
    learning_rate
)

damage_scores = []

for prompt in other_domain_probe_prompts:
    old_loss = probe_loss(student, prompt)
    new_loss = probe_loss(virtual_student, prompt)
    damage_scores.append(new_loss - old_loss)

fragile_prompts = topk(damage_scores)

# 关键：只保存 prompt，不复用很久以前的 completion
fresh_replay_rollouts = student.generate(fragile_prompts)
fresh_teacher_signals = routed_teachers.score(
    fresh_replay_rollouts
)

total_loss = (
    source_loss
    + replay_weight * mopd_loss(
        fresh_replay_rollouts,
        fresh_teacher_signals
    )
)

total_loss.backward()
optimizer.step()
```

这里有一个很实用的省算力技巧：

只需要对当前 Math batch backward 一次。得到 virtual update 之后，对 replay 候选做前向计算就可以排序，不需要给每个候选单独求梯度。

3.2 一个实际例子

一次 Math update 对 IF probe 的预计损失变化为：

IF prompt 类型	预计 loss 增加
“只能回答一个单词”	+0.22
“严格输出 JSON”	+0.18
“不要解释，只输出数字”	+0.11
普通问答	+0.01

那么系统不需要随机回放 64 条 IF prompt，只选择前三类最容易受损的 prompt。

接下来：

1. 用当前学生对它们重新 rollout；
2. IF teacher 重新提供 token-level 信号；
3. 将其作为 10%–20% 的保护 batch；
4. 与 Math batch 联合更新。

这种 replay 的作用不是“重新学 IF”，而是：

在吸收 Math teacher 知识时，给参数更新增加一组当前最需要保护的边界条件。

3.3 不要真的“每个 step 都 replay”

文档里“每次更新后回放”是概念表达。实际实现推荐：

- 每 50–200 个 step 做一次影响检查；
- 或者只有当预计伤害超过阈值时触发；
- replay token budget 不超过总 budget 的 10%–20%；
- 先根据影响矩阵确定最脆弱的目标领域，再在该领域内部选 prompt。

例如：

```
Math update
  ↓
A[Math, IF] < -threshold
  ↓
在 IF probe bank 中进行 fragile ranking
  ↓
取 Top-K prompt 做 on-policy replay
```

这样比在所有领域、所有样本上搜索便宜得多。

3.4 它与 MIR 的重合到底有多大？

重合确实不小。

MIR 的核心就是通过 virtual update 找到“即将受到最大干扰的 memory samples”，再进行 replay。MIR

所以你不能把下面这句话当成主要创新：

“我们预测 loss 增加最大的样本，然后 replay。”

这已经很接近 MIR。

你需要强调的区别是：

- MIR 通常保存并回放历史训练样本；
- EA-MOPD 保存的是 prompt，不长期复用旧 completion；
- replay 时由当前 student 重新 on-policy rollout；
- routed teacher 重新给 token-level distillation signal；
- replay 由跨领域影响矩阵触发；
- 目标不是顺序任务中的旧任务遗忘，而是并行多教师蒸馏中的跨能力副作用。

即使这样，Fragile replay 也不太适合单独作为整篇论文的主创新。它更适合作为 Externality-aware allocator 的保护模块。

四、Progress-aware sampling 具体怎么做？

Progress-aware 解决的是另一个问题：

gap 大不代表“值得继续投入算力”。

例如两个 Math bucket：

Bucket	当前 gap	消耗 token	gap 下降
Math-hard	0.90	100k	0.01
Math-medium	0.50	20k	0.08

如果只看 raw gap，会选 Math-hard。

但单位百万 token 的实际进步为：

$$LP_{\text{hard}} = \frac{0.01}{0.1} = 0.1 [LP_{\text{medium}} = \frac{0.08}{0.02} = 4.0]$$

Math-medium 的学习效率高出很多。

这时更合理的策略是：

- 先训练仍然学得动的 Math-medium；
- 暂时降低 Math-hard 的预算；
- 保留少量 exploration，避免永远放弃 hard bucket。

4.1 Learning progress 怎么计算？

对每个领域或 bucket 维护 held-out probe loss：

$$E_i(t)$$

当训练领域 i 一段时间后：

$$LP_i(t) = \frac{E_i(t_{\text{before}}) - E_i(t_{\text{after}})}{\text{tokens consumed by } i}$$

然后做 EMA：

$$\overline{LP}_i(t) = \rho \overline{LP}_i(t-1) + (1 - \rho) LP_i(t)$$

建议不要一上来做到单样本粒度。先使用：

- 三个领域；
- 或每个领域按 difficulty/length 分成 3 桶；
- 总共 9 个 sampling arms。

4.2 Forgetting speed 怎么计算？

假设 IF 上一次完成维护时 probe loss 是 0.20。训练了一段 Math 后，IF loss 上升到 0.26。

那么：

$$F_{\text{IF}} = \frac{0.26 - 0.20}{\text{期间其他领域消耗的 token}}$$

更一般地：

$$F_i(t) = \frac{\max(0, E_i(t) - E_i(t_i^{\text{last-trained}}))}{\text{elapsed non-}i \text{ tokens}}$$

它回答的是：

如果暂时不给领域 i 训练预算，它退化得有多快？

于是会出现三种典型领域：

情况	解释	策略
gap 大, progress 高	有空间而且学得动	多采样
gap 大, progress 接近 0	可能进入平台期	减少预算、保留探索
gap 小, forgetting 高	当前不错但非常脆弱	周期性维护

4.3 一个简单可用的 sampling score

可以定义：

$$S_i = \alpha G_i \max(LP_i, 0) + \beta F_i + \kappa \sqrt{\frac{\log t}{n_i + 1}} - \gamma C_i$$

其中：

- G_i : 剩余 gap;
- LP_i : 单位 token 学习进度;
- F_i : 遗忘速度;
- 最后一项 UCB bonus 用于探索;
- C_i : rollout 成本。

这比单纯按 raw gap 采样更合理:

- 不是只看“还差多少”;
- 还看“给它 token 是否真的有效”;
- 以及“不训练它会不会迅速退化”。

4.4 为什么这个方向单独发文章偏弱?

因为动态数据混合、按训练收益重加权已经有较多工作:

- DoReMi 使用 proxy model 和 Group DRO 学习预训练数据比例; [DoReMi](#)
- DoGE 使用双层优化估计领域泛化收益, 并显式关注领域依赖; [DoGE](#)
- Open-MOPD 已经使用 gap-aware budget;
- CaMOPD 也使用 teacher-student gap 做样本筛选和 recovery/preservation 调度。 [CaMOPD](#)

所以如果方法只是:

“我们根据 gap 的下降速度调整 sampling ratio。”

审稿人很可能会认为是一个合理但增量式的 scheduler。

Progress-aware 更适合作为:

- Externality-aware controller 中的 w_j ;
- 与 raw-gap sampling 的对照实验;
- influence estimation 失败时的后备方向。

五、三个模块合起来是什么样?

完整训练循环可以写成:

```
初始化 student、多个 teachers、各领域 prompt pool
初始化各领域 probe bank
初始化影响矩阵 A

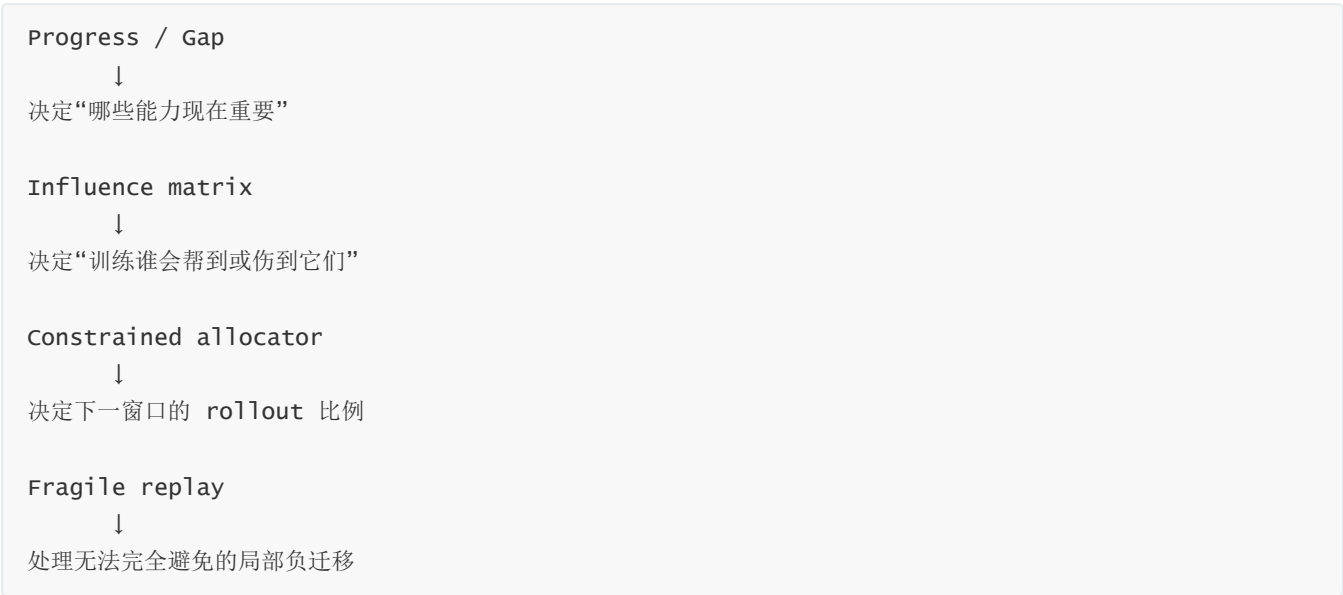
for each training window:

    1. 使用当前 student 刷新少量 probe rollout

    2. 估计 DxD 影响矩阵
        A[i,j] = 训练 i 对 probe j 的预计收益
```

- 3. 更新状态
gap[j]
learning_progress[j]
forgetting_rate[j]
rollout_cost[j]
- 4. 求下一窗口 rollout 比例 p
最大化综合收益
同时约束任何领域不能严重退化
- 5. 在接下来的 H 个 step 中：
按 p 选择领域
从对应 prompt pool 抽 prompt
当前 student rollout
对应 teacher 提供信号
执行 Open-MOPD 更新
- 6. 如果 A[source,target] 低于阈值：
在 target probe bank 内找最脆弱 prompt
重新 on-policy rollout
加入少量 protective replay

核心关系是：



六、和已有工作的边界必须怎么写？

已有工作	与你的重合	必须保住的差异
Open-MOPD	动态 budget、gap、reward refresh	你控制 rollout 前的领域比例，并建模跨领域负外部性
CaMOPD	recovery/preservation、gap-based selection	你不是固定交替，而是在线估计谁伤害谁

已有工作	与你的重合	必须保住的差异
GradAlign	用 validation gradient alignment 选择 RL 数据	你必须维护有方向的 $D \times D$ 矩阵，而不是聚合成单个 validation gradient
DoGE	域重加权、域间依赖	你是在线、非平稳、on-policy、多教师、带 no-regression constraint
DoReMi	学习 data mixture	它偏两阶段静态预训练比例；你是训练过程中持续更新
MIR	预测受损样本并 replay	retrieval rule 本身不能算新；新意在于 on-policy prompt replay 和 MOPD coupling

其中最危险的重合是 GradAlign。

GradAlign 用 candidate policy gradient 与 validation policy gradient 的 cosine 选择在线 RL 数据。[GradAlign](#)

如果你的方法最后变成：

$$\text{score}_i = \cos(g_i, \sum_j h_j)$$

然后选择得分最高的数据，那么很容易被评价为“GradAlign applied to MOPD”。

你必须保留：

1. 不聚合目标领域，而是维护 $D \times D$ 有向矩阵；
2. 显式展示非对称关系，例如 Math→IF 有害，但 IF→Math 不一定有害；
3. 优化 worst-domain/no-regression，而不仅是平均 validation loss；
4. 分配的是 pre-rollout compute，而不仅是从已有 rollout 中筛选数据；
5. 证明矩阵随训练阶段变化，是动态而不是固定 data mixture。

近期工作已经观察到多教师蒸馏中存在跨领域迁移与 seesaw，甚至领域路由也不能完全隔离 teacher influence，这正好可以作为你的问题动机。[Every Coin Has Two Sides](#)

七、在正式实现前，先做这个“生死实验”

不要立即实现完整 scheduler。先回答最根本的问题：

Math update 对 Code/IF 的影响是否真实存在，并且能否被一步梯度预测？

实验设置

使用 Math、Code、IF 三个领域，在训练过程取 5 个 checkpoint。

对每个 checkpoint：

1. 每个领域抽 10 个 calibration batch；
2. 分别计算一次领域梯度 g_i ；
3. 预测 $A_{ij} = \eta h_j^\top g_i$ ；
4. 在 LoRA clone 或 virtual weights 上真正执行一步更新；

5. 重新计算三个 probe loss;

6. 得到真实影响:

$$A_{ij}^{\text{actual}} = L_j(\theta) - L_j(\theta'_i)$$

然后比较:

- predicted sign 与 actual sign 是否一致;
- Spearman correlation;
- 不同 checkpoint 的矩阵是否变化;
- off-diagonal 项是否显著非零;
- cosine、dot product、last-layer gradient 哪个预测更好。

我建议的工程性 go/no-go 标准是:

- off-diagonal 负影响能够稳定复现;
- 符号预测准确率达到约 65%–70%;
- Spearman 至少约 0.3–0.5;
- 简单的安全约束分配能提高 worst-domain performance。

这些不是文献规定的标准,而是你决定是否继续投入的实用阈值。

如果发现:

- 跨域损害很弱: Externality 主线不成立;
- 损害存在但 gradient 预测不准: 换 virtual-update estimator;
- 预测准确但控制器没收益: 检查矩阵更新频率和 rollout 成本归一化;
- progress 信号有效但 externality 无效: 退回 Progress-aware 方向。

八、我的最终可行性判断

方向	工程可行性	单独作为论文创新	适合的定位
Transfer-aware domain matrix	高	中—高, 取决于诊断结果	主方法
Fragile-capability replay	很高	偏低, 和 MIR 接近	保护模块
Progress-aware sampling	很高	低—中, 类似 scheduler 较多	辅助信号/基线

最建议的论文版本是:

学习非平稳的跨领域影响矩阵, 根据净收益和 no-regression constraint 动态分配 student rollout; 当不可避免的负迁移出现时, 通过 fresh on-policy fragile prompt replay 保护受损能力。

而不是把三者都包装成同等重要的贡献。

另外, 暂时不要同时引入“异步 rollout”。先在同步训练下证明 externality estimator 和 allocator 有效; 否则实验失败时, 你无法判断是影响矩阵不准, 还是 asynchronous staleness 造成的。等主方法成立后, 再把异步系统中的 rollout age/staleness 加进矩阵, 才会成为自然扩展。

Q3:

你看一下这个文件中的内容，是我某次和你的对话历史，我现在最大的担忧是，其中需要使用calibrate和probe 两个不同的数据，然后，我的理解是，算出student在这两个数据上，需要变化的梯度，然后判断梯度方向是否一致或者简单理解，更新方向是否一致？

所以，我最大的问题是，如何确保这两个数据的质量呢？calibrate比较好理解，就是当前需要使用的batch 数据，而probe我理解的是，类似于验证集，最后模型应该在这个数据集上有一个好的效果。但是如何确保这两个数据的质量呢？或者如何确保probe确实可以代表我们希望的，模型更新的方向呢？

既然probe是希望的模型更新的方向？为什么不直接使用这部分的数据进行训练呢？这个我还是不太理解？

第二，如何确保梯度方向是否一致确实代表了能力的变化？举个例子，如果分别使用Math数据和IF数据算出了梯度，然后发现两个乘积为负值，理论上，如果只是用Math的数据，会导致模型的更新方向与IF数据相反，这就代表了使用这个math数据进行训练，就会导致模型在IF数据上效果变差嘛？

请你帮我打消上面的疑虑，同时，你也可以看看类似的论文中有没有遇到相同的问题？它们是如何解决的呢？？

你的担忧完全成立，而且它不是一个靠解释就能消失的问题。准确结论是：

Probe 不能天然代表“正确的能力方向”。它只是我们人为定义的能力测量器。

方法是否成立，取决于 probe 是否具有代表性、probe loss 是否与真实能力指标相关，以及梯度预测能否通过真实更新验证。

因此，正式方案不应该只有 calibration 和 probe 两份数据，而应该有三层：

数据	作用	是否参与训练
Source/calibration batch	产生候选更新	是
Controller probe	判断候选更新可能帮助或伤害什么能力	不直接训练，但会被间接优化
Audit/test	验证 probe 判断是否真的对应能力变化	绝不能影响采样决策

我甚至建议把“calibration batch”改名为“source batch”或“candidate batch”，避免与传统模型校准混淆。

一、Probe 不是“希望的参数更新方向”

假设：

$$g_{\text{Math}} = \nabla_{\theta} L_{\text{Math}}^{\text{source}} [h_{\text{IF}} = \nabla_{\theta} L_{\text{IF}}^{\text{probe}}]$$

其中：

- g_{Math} ：当前 Math batch 希望怎样改变模型；
- h_{IF} ：怎样改变模型可以降低当前 IF probe loss。

如果用 SGD 执行 Math 更新：

$$\Delta \theta_{\text{Math}} = -\eta g_{\text{Math}}$$

IF probe loss 的一阶变化为：

$$\Delta L_{\text{IF}} \approx h_{\text{IF}}^{\top} \Delta \theta_{\text{Math}} = -\eta h_{\text{IF}}^{\top} g_{\text{Math}}$$

所以：

- $h_{\text{IF}}^{\top} g_{\text{Math}} > 0$ ：预计 IF probe loss 下降；
- $h_{\text{IF}}^{\top} g_{\text{Math}} < 0$ ：预计 IF probe loss 上升；
- 接近 0：Math 更新对这个 IF probe 的局部影响不明显。

但这里的结论只能读成：

当前这一小步 Math 更新，预计会让当前 IF probe 上的这个 surrogate loss 上升。

不能直接读成：

Math 训练会导致模型总体 IF 能力下降。

中间还隔着三道验证：

梯度内积

↓

是否预测真实 virtual-update loss 变化

↓

probe loss 变化

↓

是否预测独立 IF benchmark 的能力变化

二、如何确保 calibration/source batch 的质量？

Source batch 的“质量”与 probe 不一样。

它不一定非得是干净的 gold data；它最重要的是忠实代表下一阶段真正会产生更新。

例如 Math source batch 应该满足：

- 来自正式 Math prompt pool，而不是特别挑选的一小组样本；
- 难度、长度、子领域比例与真实 rollout 分布接近；
- rollout 由当前 student 生成，避免严重 on-policy staleness；
- teacher 路由正确；
- teacher signal、reward 或 token probability 没有明显错误；
- 不被极少数超长样本或超大梯度样本支配；
- 按实际消耗的 rollout/teacher tokens 归一化。

如果实际训练池中有噪声，不应该为了计算影响矩阵而把 source batch 过度清洗，否则估计的是“理想 Math 数据”的影响，而不是“真正将要训练的数据”的影响。

降低 source batch 方差可以采用：

1. 每个领域不只抽一个 batch，而是抽 4-10 个小 batch；
2. 按难度、长度和子领域分层采样；
3. 对梯度范数做 clipping；

4. 报告多批次影响的均值和置信区间；
5. 如果影响符号不稳定，就暂时将该矩阵元素视为未知，而不是强行判正负。

三、如何确保 probe 代表我们想保护的能力？

这是整个方法最核心、也最容易被审稿人攻击的地方。

Probe 的质量至少要通过三类 validity。

3.1 内容代表性：它覆盖了能力的哪些方面？

不能简单地从 IF 数据集中随机抽 100 条，然后就声称代表“Instruction Following”。

应该先定义 capability blueprint。

例如 IF 可以拆为：

- 输出格式约束：JSON、表格、单词；
- 长度约束；
- 风格约束；
- 禁止项与否定指令；
- 多约束组合；
- 指令冲突与优先级；
- 长上下文中的约束保持。

Math 可以拆为：

- 算术；
- 代数；
- 几何；
- 概率；
- 应用题；
- 短推理与长推理；
- 不同难度。

Code 可以拆为：

- 算法实现；
- debugging；
- API 使用；
- 代码补全；
- 单元测试修复；
- 不同语言和长度。

然后按照你最终希望模型具备的能力分布设置 probe 权重。

如果最终部署分布未知，可以：

- 主实验使用 capability-balanced 权重；
- 额外报告 uniform、真实数据比例、worst-capability 三种权重；
- 不要过早把所有 probe 聚合成一个标量。

Skill-It 的结果恰好说明“随便选一个语义分组”不一定能构成有效技能：他们发现有些数据分组产生几乎完全连接或几乎空的技能图，对数据选择帮助很小。因此技能划分本身也需要实证验证。 [Skill-It](#)

3.2 测量有效性：probe loss 是否真的对应能力？

这是第二个容易忽略的问题。

假设 IF probe 使用的是对 IF teacher 的 token-level KL：

$$L_{\text{IF}}^{\text{probe}} = D_{\text{KL}}(\pi_{\text{teacher,IF}} \parallel \pi_{\theta})$$

那么它严格测量的是：

Student 在这些 trajectories 上是否更接近 IF teacher。

它不一定等于真实的 instruction-following 能力。

可能出现：

- KL 上升，但严格格式准确率没变；
- KL 下降，只是更模仿 teacher 的措辞；
- teacher 本身在某些指令上有偏差；
- token-level loss 变化没有跨过最终答案的决策边界。

因此最好使用“双层测量”：

可微分 controller loss

用于计算梯度，例如：

- MOPD teacher loss；
- token-level KL；
- policy-gradient surrogate；
- supervised NLL。

独立能力指标

只用于验证，例如：

- Math：exact match、数学 verifier；
- Code：unit tests、pass@k；
- IF：constraint checker、IFEval 类型指标；
- 通用能力：独立 benchmark accuracy。

只有当：

$$\Delta L_{\text{probe}}$$

和

$$\Delta M_{\text{audit}}$$

在多个 checkpoint、batch 和随机种子上具有稳定相关性，才能说 probe loss 是有意义的能力代理。

如果没有这种相关性，那么问题不在影响矩阵，而是 probe loss 选错了。

3.3 统计可靠性：换一批 probe，方向还一致吗？

一个 probe gradient 可能有很大采样噪声。

建议把 probe bank 随机分成若干子集：

$$P_j^{(1)}, P_j^{(2)}, \dots, P_j^{(K)}$$

分别计算：

$$h_j^{(k)} = \nabla L(P_j^{(k)})$$

然后检查：

- 各 probe 子集梯度与平均梯度的 cosine；
- $g_i^\top h_j^{(k)}$ 的符号一致率；
- bootstrap confidence interval；
- 不同 rollout seed 下的稳定性；
- 不同 checkpoint 下是否重复出现。

例如 10 个 probe resample 中：

- 9 次 Math→IF 为负：可信度较高；
- 5 次正、5 次负：应该视为“不确定”；
- 平均值为负但置信区间跨过 0：不能据此阻止 Math rollout。

控制器最好维护：

$$\mu_{ij}, \quad \sigma_{ij}$$

并使用保守估计：

$$A_{ij}^{\text{LCB}} = \mu_{ij} - z\sigma_{ij}$$

而不是只使用一次梯度乘积。

四、既然 probe 很高质量，为什么不直接拿来训练？

答案是：

如果 probe 足够大、允许训练而且有高质量标签，当然可以把其中一部分加入训练；但必须再留出一部分不训练的数据作为新 probe。

这不是“probe 数据神圣不可训练”，而是必须保留独立测量信号。

4.1 直接训练会失去独立性

假设总是用同一批 100 条 IF probe 指导 rollout allocation。

虽然没有直接对它们执行 optimizer step，但训练数据一直是根据它们筛选的，所以它们实际上已经成为一种 meta-training signal。

随着训练推进，模型和 controller 可能逐渐对这 100 条 prompt 的梯度结构过拟合。

如果再直接训练它们，就更无法判断：

- 模型学会了总体 IF；
- 还是只记住了这 100 条指令和相似轨迹。

所以需要有一个完全不参与控制的 audit set。

4.2 小 probe 缺少训练多样性

Probe 的特点通常是：

- 小；
- 干净；
- 平衡；
- 覆盖关键能力。

而正式训练池的特点是：

- 大；
- 多样；
- 可以产生大量新轨迹；
- 能覆盖更多语言表达和推理路径。

利用 probe 挑选与它方向一致的大规模数据，可以理解为：

用少量高质量数据定义目标，用大规模多样数据完成学习。

这也是 validation-based reweighting 的经典设定。Ren 等人的方法明确假设一小份干净、无偏且与评估一致的验证集，用它给大规模、可能有噪声的训练样本赋权。[Ren et al., 2018](#)

4.3 GradAlign 直接实验过这个问题

GradAlign 比较了：

- 直接在小型 validation set 上训练；
- 使用 validation gradient 筛选大规模训练问题。

论文报告，直接训练能在 validation set 自身取得更高表现，但对独立测试集泛化更差；作者将其解释为对小型 validation trajectories 的过拟合，而 gradient-guided selection 保留了训练数据多样性。[GradAlign](#)

所以一个合理的数据划分是：

80%–90%: source/train pool
5%–10%: controller probe
5%–10%: untouched audit

更稳妥的是 audit 数据来自不同数据源、不同年份或独立 benchmark，而不只是同一个数据集随机切分。

五、负梯度内积究竟说明了什么？

5.1 对理想的小步 SGD，它的数学含义是明确的

例如：

$$g_{\text{Math}} = (1, 0) [h_{\text{IF}} = (-0.2, 0.1)]$$

则：

$$h_{\text{IF}}^\top g_{\text{Math}} = -0.2$$

取 $\eta = 0.01$ ，预测：

$$\Delta L_{\text{IF}} \approx -\eta(-0.2) = +0.002$$

也就是 IF probe loss 预计上升 0.002。

这个推导没有问题。

但如果 probe loss 自身的标准误差是 0.005，那么这个结果在统计上没有意义。符号为负不等于影响显著。

5.2 它不是总体能力下降的充分条件

以下任何情况都可能导致“负内积但没有真实能力下降”：

- IF probe 不具有代表性；
- probe batch 太小；
- token-level loss 上升但最终答案没变；
- 一步更新很小，指标变化处于噪声范围；
- 学习率较大，高阶曲率项不可忽略；
- 影响只发生在一个 IF 子能力；
- 一步产生局部损害，但后续联合训练迅速修复；
- Math 和 IF 的两个随机 minibatch 偶然冲突。

反过来也可能出现：

- 梯度内积接近零；
- 但连续 100 个 Math step 后 IF 明显下降。

因为长期非线性效应不是一步 Taylor 展开能够完整预测的。

5.3 如果使用 Adam，不能简单使用原始梯度内积

这是实现时非常重要的一点。

AdamW 的实际更新并不是：

$$\Delta\theta = -\eta g$$

它还受到：

- 一阶动量；
- 二阶矩预条件；
- weight decay；
- gradient clipping；
- learning-rate schedule

影响。

因此更准确的估计应使用候选 batch 在当前 optimizer state 下产生的实际更新向量：

$$\Delta\theta_i^{\text{opt}} = \text{OptimizerStep}(g_i, \text{optimizer state})$$

然后预测：

$$\Delta L_j \approx h_j^\top \Delta\theta_i^{\text{opt}}$$

如果：

$$h_j^\top \Delta\theta_i^{\text{opt}} > 0$$

才表示目标 loss 预计上升。

最稳妥的办法仍然是 virtual update：复制 LoRA 参数和 optimizer state，真的模拟一次候选 AdamW 更新，然后重新测 probe loss。

六、相关论文实际上也没有“理论解决”probe 代表性

它们使用的是假设、数据设计和 held-out 验证。

工作	如何处理这个问题	仍然存在的限制
Ren et al.	假设一小份干净、无偏、与评估一致的验证集	如果验证集有偏，权重也会有偏
Skill-It	训练技能和评价技能使用独立数据，并用真实训练若干步后的 validation loss 建图	技能划分不合理时，技能图没有价值
DoGE	明确定义 target domain，通过 target validation loss 学 domain weights	最终方向完全由 target-domain 定义决定
GradAlign	小型 trusted validation、独立 held-out test、周期性重算当前策略梯度	论文明确承认 validation-test mismatch 会选择错误问题
TracIn	用梯度点积和多个训练 checkpoint 近似训练样本影响	这是影响估计器，不是能力变化证明

工作	如何处理这个问题	仍然存在的限制
PCGrad/GEM	把负梯度内积视为任务冲突或潜在遗忘并约束更新	仍然通过独立任务性能验证是否真的减少遗忘

DoGE 的原理就是上调与目标领域梯度内积较大的训练领域，但它最终仍要在 perplexity 和下游任务上验证泛化。[DoGE](#)

TracIn 也把梯度点积当成一阶 influence approximation，并通过多个 checkpoint、选择部分层等方式提高可扩展性；它没有把单次负点积直接宣称为能力下降。[TracIn](#)

PCGrad 和 GEM 则更保守：检测到潜在冲突后修改或约束更新，最终仍看真实任务性能，而不是把“负内积”本身作为最终结论。[PCGrad](#)、[GEM](#)

特别值得注意的是，GradAlign 直接把 probe mismatch 列为限制：如果 validation 代表 MMLU，而最终目标是 AIME，方法就可能朝错误方向选择数据。这说明文献也承认，probe 代表性不是梯度方法本身能够解决的。[GradAlign](#)

七、我建议你使用“三级验证”，而不是直接相信梯度

对每个 source i 和 target j ，依次验证：

第一级：Gradient prediction

$$\widehat{\Delta L}_{ij} = h_j^\top \Delta \theta_i^{\text{opt}}$$

它便宜，用于在线估计。

第二级：Virtual-update consequence

模拟真实 optimizer step：

$$\Delta L_{ij}^{\text{virtual}} = L_j(\theta'_i) - L_j(\theta)$$

它验证梯度近似是否正确。

第三级：Capability audit

在独立 audit 数据上评价：

$$\Delta M_{ij}^{\text{audit}} = M_j(\theta'_i) - M_j(\theta)$$

它验证 probe loss 是否对应真实能力。

研究问题应当拆成两个相关性：

$$\widehat{\Delta L}_{ij} \longleftrightarrow \Delta L_{ij}^{\text{virtual}}$$

以及：

$$\Delta L_{ij}^{\text{virtual}} \longleftrightarrow \Delta M_{ij}^{\text{audit}}$$

只有两层都成立，才能用 gradient externality 控制 rollout。

八、最适合你现在做的生死实验

暂时不要实现完整动态 sampler。先进行一个小规模 validity study。

在 5 个训练 checkpoint 上，对 Math、Code、IF 分别：

1. 抽取 10 个 source batch；
2. 抽取多个不同的 probe 子集；
3. 计算 gradient inner product；
4. 模拟一次真实 AdamW/LoRA virtual update；
5. 测量 probe loss 前后变化；
6. 测量独立 audit benchmark 的变化；
7. 重复不同 rollout seed。

最终报告四个指标：

- Gradient $\rightarrow$ virtual loss 的 sign accuracy；
- Gradient $\rightarrow$ virtual loss 的 Spearman correlation；
- Virtual probe loss $\rightarrow$ audit metric 的相关性；
- 不同 probe 子集上的符号一致率。

如果结果是：

Math $\rightarrow$ IF:
80% source batch 显示负影响
不同 probe 子集符号一致
virtual update 后 IF probe loss 显著上升
独立 IF audit 指标也下降

这时才能有底气说：

当前阶段的 Math 更新会产生可测量的 IF 负外部性。

如果只是：

某一个 Math batch 与某一个 IF batch 的梯度内积为负

那最多只能说：

观察到一次局部 minibatch gradient conflict。

不能上升到能力层面。

最终，你不需要相信“梯度方向天然代表能力”。更科学、也更容易形成论文的方法定位是：

梯度内积只是低成本预警器；virtual update 是局部因果检查；独立 audit set 才是能力证据。

如果 probe 很差，这个方法确实会失败；任何类似论文都无法绕过这一点。你真正可以贡献的，恰恰是把 probe reliability、影响不确定性和真实能力验证明确纳入 MOPD controller，而不是把一个负梯度乘积直接当成能力退化。

Q4:

这个idea有没有类似的，你是怎么想到的这个idea呢？是不是有类似的论文提出了这个idea，只是不是MOPD这个方向

先说结论：有，而且有些工作非常接近。尤其是 Skill-It，已经包含了“训练技能 i 对评估技能 j 的影响矩阵 + 根据矩阵动态分配数据”这个骨架。

因此，我需要修正前面比较乐观的判断：

单纯提出 $D \times D$ 影响矩阵、计算 $g_i^\top h_j$ 、再动态采样，这个方法本身不能算很强的新颖性。

真正可能形成 MOPD 论文的，是“多教师、on-policy、带负外部性、非平稳、按 rollout 成本约束”的特殊问题和解决方案。

1. 最相似的工作：Skill-It

[Skill-It! A Data-Driven Skills Framework for Understanding and Training Language Models](#) 几乎已经提出了一个“技能影响矩阵”。

它定义：

$$A \in \mathbb{R}^{k \times m}$$

其中：

$$A_{ij}$$

表示：

在训练技能 i 的数据后，对评估技能 j 有多大帮助。

这与我们前面说的：

```
训练 Math → 影响 Math
训练 Math → 影响 Code
训练 Math → 影响 IF
```

在概念上高度相似。

Skill-It 学习矩阵的方式也非常直观：

1. 复制一个初始模型；
2. 在技能 i 上训练 H 步；
3. 测量所有评估技能 j 的 held-out validation loss；
4. 将 loss 变化记录为 A_{ij} ；
5. 对每个 i 重复，得到完整矩阵。

它的近似方法就是：

$$A_{ij} \propto L_j(f) - L_j(f_{H,i})$$

这与我们说的 virtual update：

$$A_{ij} = L_j(\theta) - L_j(\theta - \eta g_i)$$

本质一样，只是：

- Skill-It 可能训练 H 步；
- 我们前面讨论的是一步或一个短训练窗口。

Skill-It 随后使用：

$$p_{t+1}^i \propto \exp\left(\eta \sum_j A_{ij} L_{\text{eval},j}(f_t)\right)$$

动态调整下一阶段的技能采样比例。

翻译成人话就是：

如果技能 i 能帮助当前还没有学好的多个技能，那么下一轮就多采样技能 i 。

这跟我们之前写的：

$$U_i = \sum_j w_j A_{ij}$$

然后调整 rollout 比例，结构上确实非常接近。

所以，Skill-It 是这个想法最大的重合风险。

2. 你的“对角线肯定为正”问题，在 DoGE 里直接出现了

[DoGE: Domain Reweighting with Generalization Estimation](#) 使用领域之间的梯度内积来调整预训练数据比例。

它定义的领域 j 得分可以写成：

$$W_j = \langle \nabla l_j, \sum_i \nabla l_i \rangle$$

展开以后：

$$W_j = \|\nabla l_j\|^2 + \sum_{i \neq j} \langle \nabla l_j, \nabla l_i \rangle$$

第一项：

$$\|\nabla l_j\|^2$$

就是对角线，它一定非负。

DoGE 对它的解释不是“跨领域迁移”，而是：

该领域自身的 gradient magnitude，也就是该领域当前的难度或剩余学习空间。

第二项：

$$\sum_{i \neq j} \langle \nabla l_j, \nabla l_i \rangle$$

才是跨领域影响。

所以你刚才的直觉完全正确：

对角线与非对角线其实代表两种不同东西，不应该混在一起解释。

这也提示我们应该修改 Externality-Aware MOPD 的定义：

- 对角线：self-learnability / learning progress；
- 非对角线：cross-domain externality。

不要把对角线也包装成“外部性”。

3. Calibration-probe gradient 来自哪里？

它最早可以追溯到 validation-based meta-reweighting。

[Learning to Reweight Examples for Robust Deep Learning](#) 的思路是：

1. 使用一个训练样本产生候选更新；
2. 模拟更新后的模型；
3. 检查 clean validation loss 是上升还是下降；
4. 如果该训练样本帮助 validation，就提高它的权重；
5. 如果伤害 validation，就降低它的权重。

一阶近似后，其核心项就是：

$$-\nabla L_{\text{validation}}^\top \nabla L_{\text{training-example}}$$

也就是我们前面说的：

$$h_j^\top g_i$$

区别是：

- Ren et al.：逐训练样本对一个 validation set；
- DoGE：训练领域对目标领域；
- 我们设想的 EA-MOPD：某个 teacher/domain 的 on-policy update 对其他 teacher capabilities。

所以“calibration gradient vs probe gradient”不是 MOPD 新产生的数学方法，而是 validation-based data reweighting 在 MOPD 中的应用。

4. 与这个想法相关的几条研究线

研究工作	已经解决了什么	和 EA-MOPD 的关系
Skill-It	学习 source-skill → target-skill 的矩阵，并动态采样	最接近整体框架
DoGE	用领域梯度 alignment 调整数据权重	最接近梯度公式
Ren et al.	virtual update + validation gradient 重加权样本	calibration/probe 思想来源
TracIn	用训练样本与测试样本的梯度点积估计影响	influence estimator 来源
PCGrad	发现任务梯度冲突后投影掉冲突部分	负 externality 的优化处理
GEM	约束当前更新不能增加历史任务 loss	no-regression constraint 来源

研究工作	已经解决了什么	和 EA-MOPD 的关系
TaskWeb	建立任务之间的有向 transfer graph	任务影响图思想
GradAlign	在线 RL 中用 validation policy gradient 筛选训练问题	与 on-policy 动态估计高度接近

具体来说：

- [TracIn](#) 使用训练点和测试点的 gradient dot product 近似前者对后者的影响；
- [PCGrad](#) 在多任务学习中检测负梯度内积并修改更新方向；
- [GEM](#) 约束当前任务更新不能增加历史任务 memory loss；
- [TaskWeb](#) 通过大量 pairwise transfer 实验构建 22 个 NLP 任务的有向迁移关系；
- 最新的 [GradAlign](#) 在在线 LLM RL 中，周期性重算 validation policy gradient，用 candidate problem gradient 与它的 cosine 选择训练题目。

尤其是 GradAlign：

$$\text{score}(x) = \cos(g_x, g_{\text{validation}})$$

与我们说的 probe-gradient 方法很接近。只不过它把 validation gradient 聚合成一个方向，而不是显式保留 $D \times D$ 的分领域关系。

5. 那么这个 idea 是怎么“想到”的？

我的推导过程是这样的。

第一步：发现 MOPD 不是多个互不相关的老师

如果 Math teacher 更新的是同一个 student 参数，那么它一定可能改变：

- Code 能力；
- IF 能力；
- General ability。

因此，“一个 teacher 的收益”不能只看自己的 teacher-student gap。

第二步：把一次更新对另一个能力的影响写出来

训练领域 i ：

$$\theta' = \theta - \eta g_i$$

考察能力 j ：

$$L_j(\theta') - L_j(\theta)$$

一阶展开：

$$L_j(\theta') - L_j(\theta) \approx -\eta h_j^\top g_i$$

于是自然得到：

用梯度内积估计一个领域的更新对另一个领域的影响。

第三步：把所有领域两两组合

三个 source domain，三个 target capability，就形成：

$$A \in \mathbb{R}^{3 \times 3}$$

第四步：用这个矩阵控制 rollout

如果 Math 对 IF 的伤害很大，就不能仅因为 Math gap 大而一直生成 Math rollout。

所以这个想法是从：

MOPD 的多目标耦合

+

一步 Taylor 展开

+

多任务梯度冲突

+

数据混合优化

自然组合出来的。

但是，这种推导非常自然，其他研究者也很容易走到同一个地方。因此文献中确实已经存在 Skill-It、DoGE、Tracln 等高度相关的方法。

准确地说：

这不是凭空原创出一个全新的数学原理，而是把已有的影响估计、技能图、动态数据混合方法迁移并重新组织到 MOPD 场景。

6. 现在还可以怎么做出真正的 MOPD 创新？

单纯的“影响矩阵”不够。必须证明 MOPD 带来了已有方法没有处理的新困难。

6.1 从静态正迁移图改为动态有符号图

Skill-It 的主要逻辑是寻找 prerequisite：

$$A_{ij} > 0$$

也就是“训练 i 是否帮助 j ”。

你的重点可以是：

$$E_{ij}(t) \in \mathbb{R}$$

其中同时包含：

- 正迁移；
- 负迁移；
- 非对称影响；

- 随训练阶段变化的影响。

例如：

训练早期：

Math $\rightarrow$ IF = +0.03

训练中期：

Math $\rightarrow$ IF = -0.08

训练后期：

Math $\rightarrow$ IF = -0.01

Skill-It 的 graph 主要在开始阶段学习，之后 sampling ratio 动态变化，但 graph 本身基本固定。

而你的矩阵可以每隔 H 步刷新：

$$E_t = \rho E_{t-H} + (1 - \rho) \hat{E}_t$$

真正的新问题是：

多教师 on-policy distillation 中，teacher interaction graph 是否高度非平稳？

6.2 不要把对角线放进 externality

建议重新定义：

自身学习价值

$$S_i(t) = \frac{L_i^{\text{probe}}(t-H) - L_i^{\text{probe}}(t)}{\text{领域 } i \text{ 消耗的 token}}$$

跨域外部性

$$E_{ij}(t) = \frac{L_j(\theta_t) - L_j(\theta_t - \eta g_i)}{C_i}, \quad i \neq j$$

并规定：

$$E_{ii} = 0$$

最终的 source utility：

$$U_i(t) = \alpha S_i(t) + \sum_{j \neq i} w_j(t) E_{ij}(t)$$

这样非常清楚：

- S_i ：训练这个领域自己能学到多少；
- E_{ij} ：训练它会给其他领域带来什么副作用。

这比直接使用一个包含对角线的 gradient matrix 更干净，也能正面回应 DoGE 的重合。

6.3 强调 MOPD 的 rollout 成本

Skill-It 和 DoGE 主要是在已有文本数据中调整 sampling mixture。

MOPD 的区别是，选择领域 i 后还需要：

- 1. Student 生成 on-policy rollout;
- 2. 对应 teacher 处理 rollout;
- 3. 获取 token-level signal;
- 4. 可能执行 reward refresh;
- 5. 长推理领域消耗更多生成 token。

因此成本不是“一条 Math 数据等于一条 IF 数据”。

应该优化：

$$\frac{\text{预测能力收益}}{\text{student rollout tokens} + \lambda \cdot \text{teacher inference tokens}}$$

也就是：

不是 data mixture，而是昂贵的 on-policy teacher-compute allocation。

这个表述比“动态采样”更具有 MOPD 特异性。

6.4 加入 capability no-regression constraint

已有方法通常最大化平均 validation performance：

$$\max_p \sum_j w_j \sum_i p_i E_{ij}$$

你的方法可以约束：

$$\sum_i p_i E_{ij} \geq -\delta_j, \quad \forall j$$

意思是：

下一窗口的 rollout 配比不能让任何一项能力的预计损失超过容忍范围。

这样目标就从：

选择最有帮助的数据

变为：

在有限 teacher/rollout 预算下，
最大化整体蒸馏收益，
同时控制每个能力的负迁移。

这更接近多教师蒸馏的核心问题。

7. 我现在对论文创新性的真实判断

方法版本	创新性判断
$D \times D$ 矩阵 + gradient inner product	低，DoGE/TracIn 已有核心公式

方法版本	创新性判断
影响矩阵 + 动态 sampling	低—中, Skill-It 非常接近
应用于 MOPD	中等, 仅场景迁移通常不够
动态 signed teacher-influence graph	中等, 有希望
加上 on-policy refresh、staleness、token cost	中—高
再加入 worst-domain/no-regression controller	更完整、较有希望

所以论文不能写成：

We are the first to model pairwise domain influence and dynamically sample domains.

这句话很可能站不住，因为 Skill-It 已经有 A_{ij} 和动态技能采样。

更安全、更准确的定位是：

Existing skill-graph and domain-reweighting methods assume offline data, static or aggregated transfer relations. We study signed, asymmetric, and non-stationary teacher-induced externalities in multi-teacher on-policy distillation, where choosing a domain incurs student rollout and teacher-inference costs.

对应的核心研究问题应当是：

在 MOPD 中，不同 teacher 引起的跨能力影响是否随 student policy 持续变化？能否在相同 rollout/teacher-token 预算下，通过动态 signed externality estimation 提高 worst-domain recovery？

这才是目前看起来最有希望、同时能与 Skill-It、DoGE、GradAlign 拉开距离的版本。